Solution: Palindromic Partitioning

Problem Statement

Given a string, we want to cut it into pieces such that each piece is a palindrome. Write a function to return the minimum number of cuts needed.

Example 1:

```
Input: "abdbca"
Output: 3
Explanation: Palindrome pieces are "a", "bdb", "c", "a".
```

Example 2:

```
Input: = "cddpd"
Output: 2
Explanation: Palindrome pieces are "c", "d", "dpd".
```

Example 3:

```
Input: = "pqr"
Output: 2
Explanation: Palindrome pieces are "p", "q", "r".
```

Example 4:

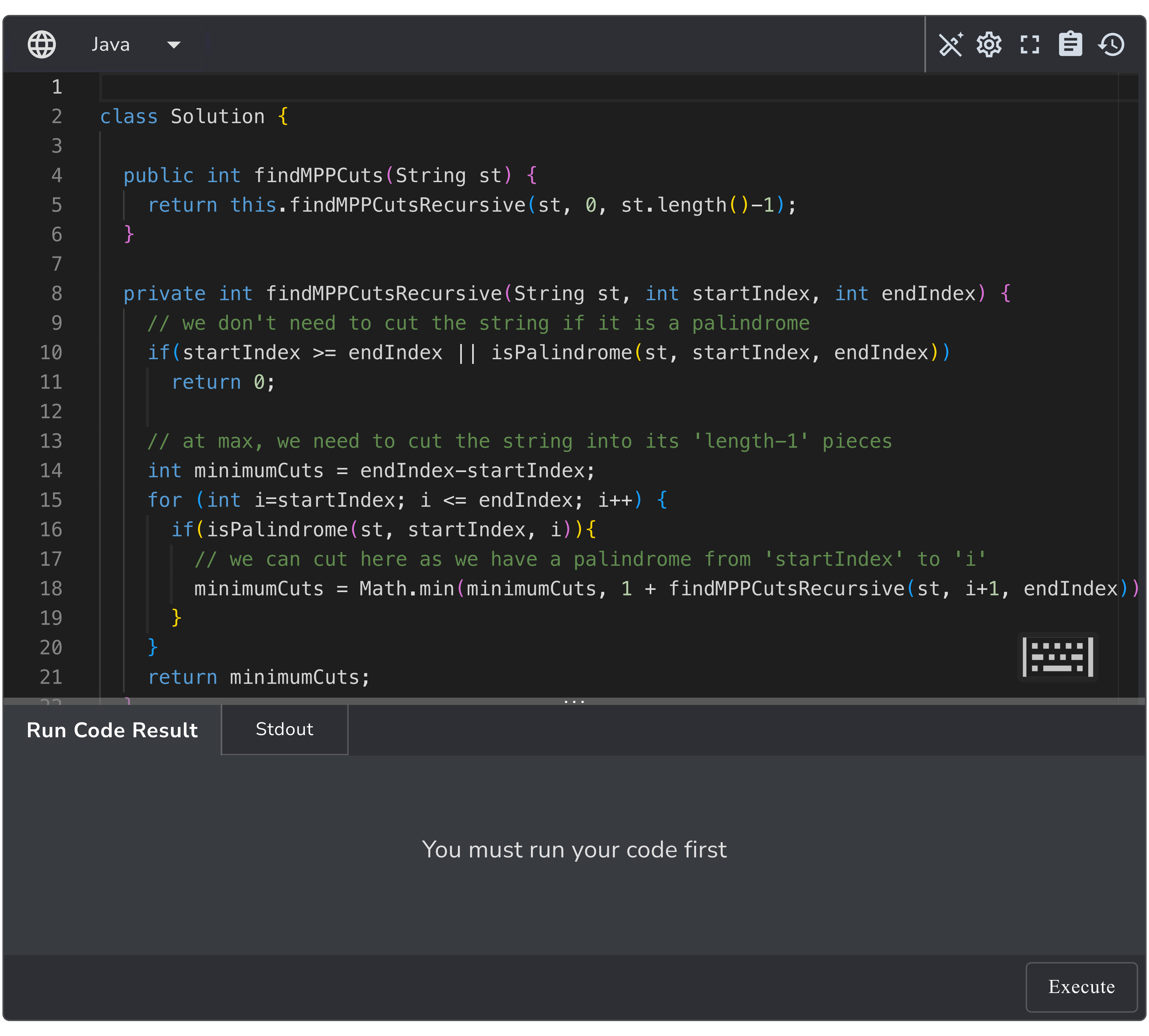
```
Input: = "pp"
Output: 0
Explanation: We do not need to cut, as "pp" is a palindrome.
```

Basic Solution

This problem follows the `"Longest Palindromic Subsequence"` pattern and shares a similar approach as that of the `"Longest Palindromic Substring"`.

The brute-force solution will be to try all the substring combinations of the given string. We can start processing from the beginning of the string and keep adding one character at a time. At any step, if we get a palindrome, we take it as one piece and recursively process the remaining length of the string to find the minimum cuts needed.

Here is the code:



```java

class Solution {

  public int findMPPCuts(String st) {
    return this.findMPPCutsRecursive(st, 0, st.length()-1);
  }

  private int findMPPCutsRecursive(String st, int startIndex, int endIndex) {
    // we don't need to cut the string if it is a palindrome
    if(startIndex >= endIndex || isPalindrome(st, startIndex, endIndex))
      return 0;

    // at max, we need to cut the string into its 'length-1' pieces
    int minimumCuts = endIndex-startIndex;
    for (int i=startIndex; i <= endIndex; i++) {
      if(isPalindrome(st, startIndex, i)){
        // we can cut here as we have a palindrome from 'startIndex' to 'i'
        minimumCuts = Math.min(minimumCuts, 1 + findMPPCutsRecursive(st, i+1, endIndex))
      }
    }
    return minimumCuts;
```

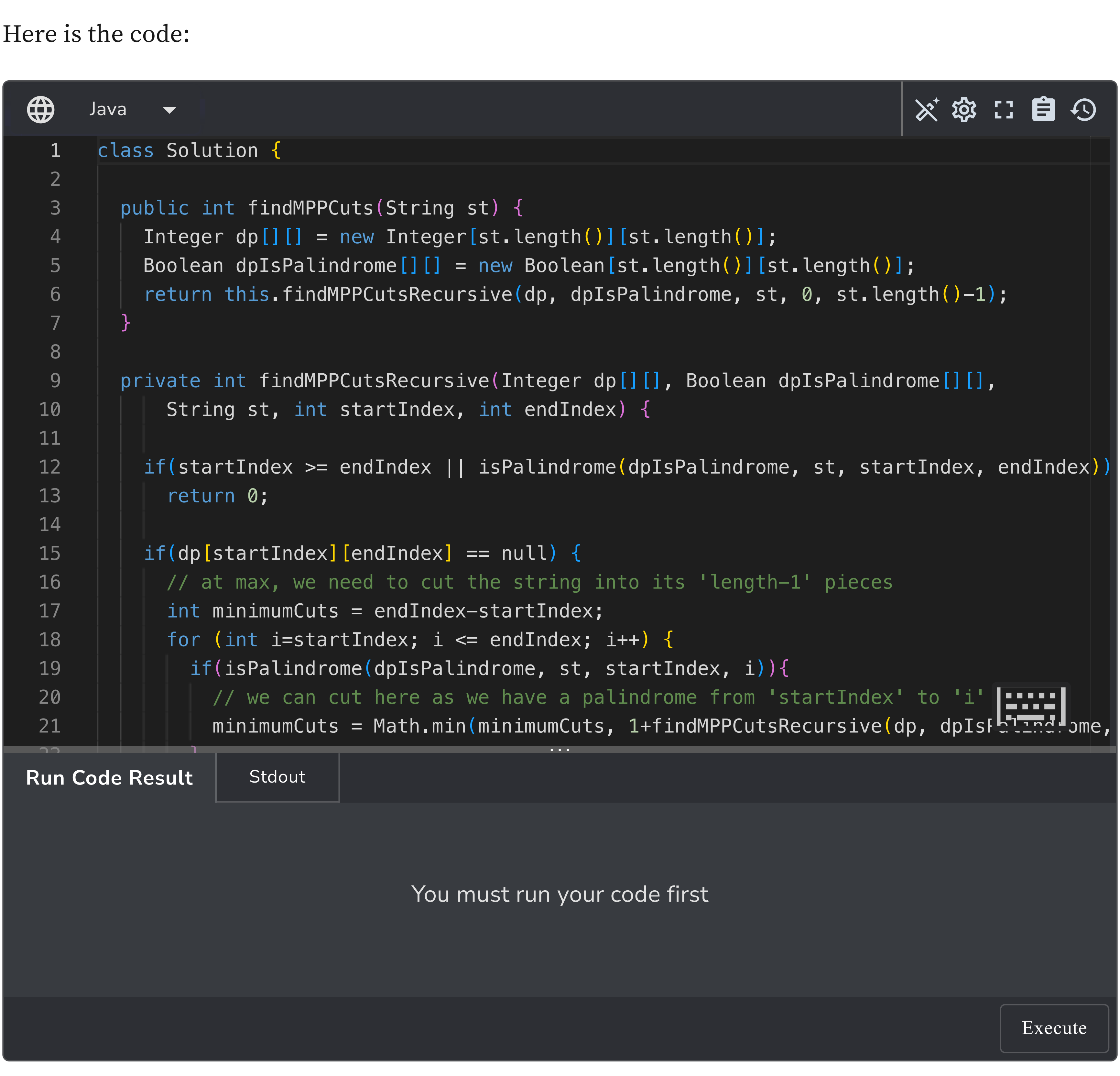
You must run your code first

The time complexity of the above algorithm is exponential $O(2^n)$, where 'n' is the length of the input string. The space complexity is $O(n)$ which is used to store the recursion stack.

Top-down Dynamic Programming with Memoization

We can memoize both functions `findMPPCutsRecursive()` and `isPalindrome()`. The two changing values in both these functions are the two indexes; therefore, we can store the results of all the subproblems in a two-dimensional array. (alternatively, we can use a hash-table).

Here is the code:



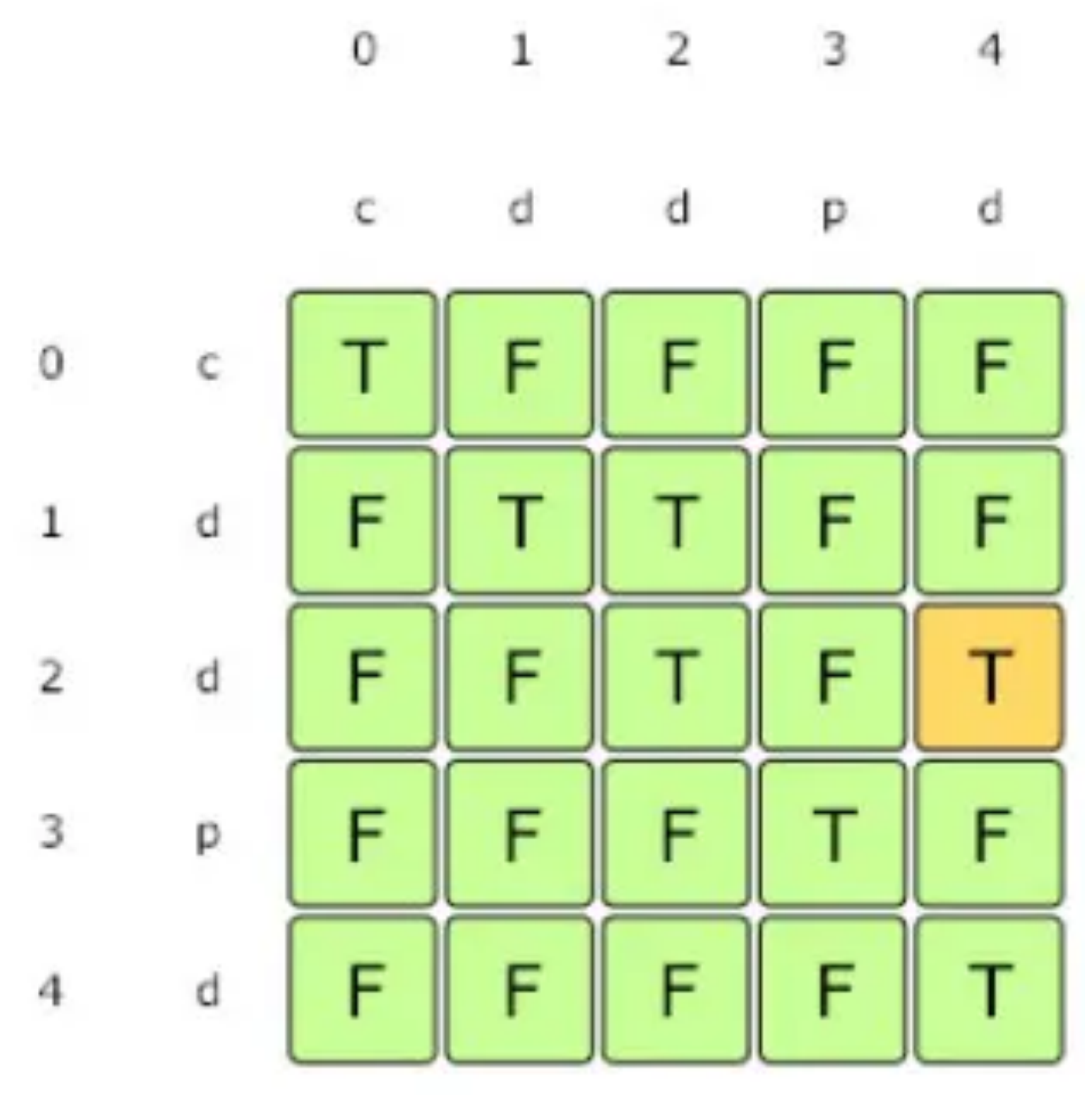
```java
class Solution {

  public int findMPPCuts(String st) {
    Integer dp[][] = new Integer[st.length()][st.length()];
    Boolean dpIsPalindrome[][] = new Boolean[st.length()][st.length()];
    return this.findMPPCutsRecursive(dp, dpIsPalindrome, st, 0, st.length()-1);
  }

  private int findMPPCutsRecursive(Integer dp[][], Boolean dpIsPalindrome[][],
      String st, int startIndex, int endIndex) {

    if(startIndex >= endIndex || isPalindrome(dpIsPalindrome, st, startIndex, endIndex))
      return 0;

    if(dp[startIndex][endIndex] == null) {
      // at max, we need to cut the string into its 'length-1' pieces
      int minimumCuts = endIndex-startIndex;
      for (int i=startIndex; i <= endIndex; i++) {
        if(isPalindrome(dpIsPalindrome, st, startIndex, i)){
          // we can cut here as we have a palindrome from 'startIndex' to 'i'
          minimumCuts = Math.min(minimumCuts, 1+findMPPCutsRecursive(dp, dpIsPalindrome,
```

You must run your code first

Bottom-up Dynamic Programming

The above solution tells us that we need to build two tables, one for the `isPalindrome()` and one for finding the minimum cuts needed.

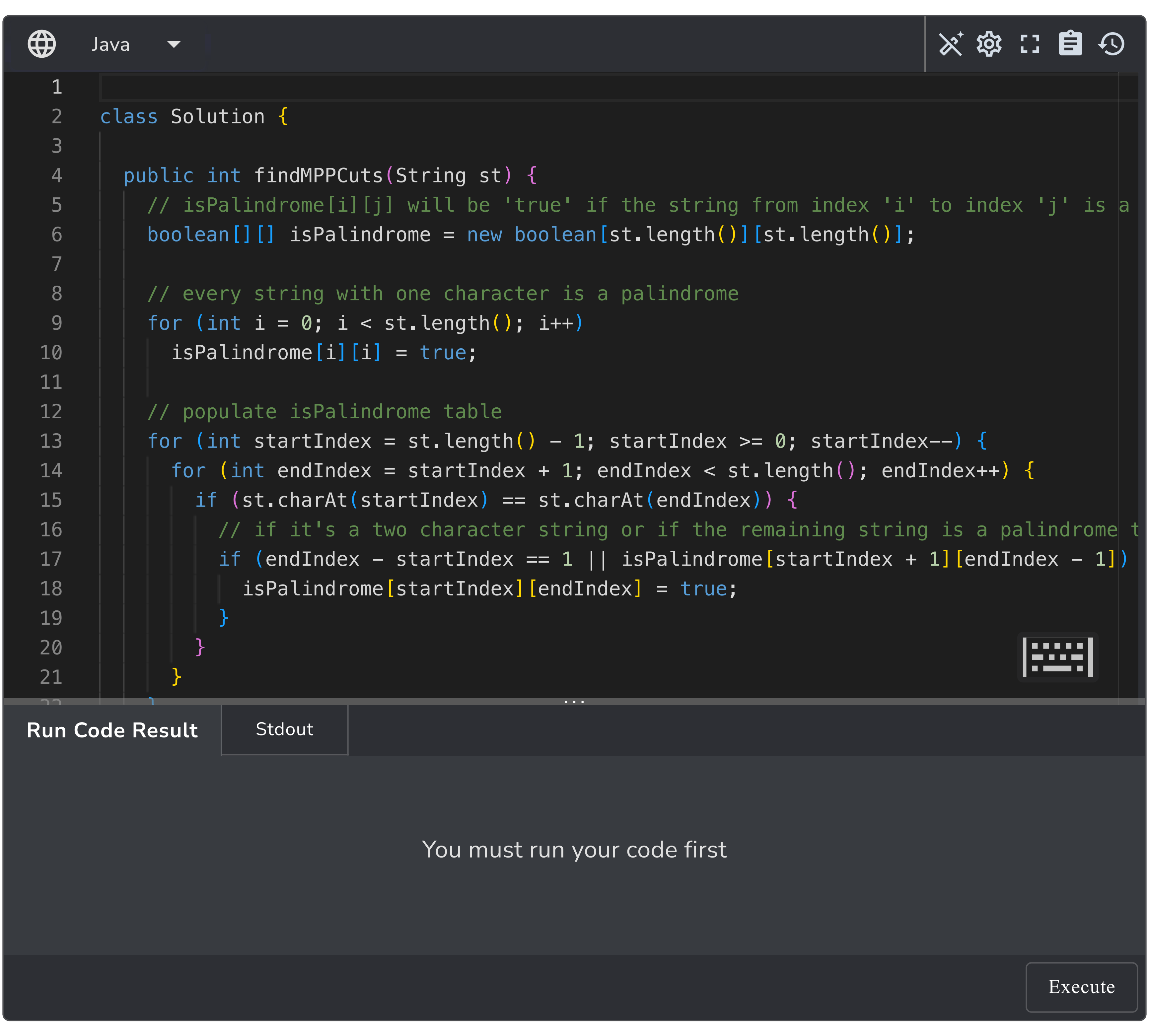
If you remember, we built a table in the Longest Palindromic Substring (LPS) chapter that can tell us what substrings (of the input string) are palindrome. We will use the same approach here to build the table required for `isPalindrome()`. For example, here is the final output from LPS for "cddpd". From this table we can clearly see that the `substring(2,4) => 'dpd'` is a palindrome:



		0	1	2	3	4
		c	d	d	p	d
0	c	T	F	F	F	F
1	d	F	T	T	F	F
2	d	F	F	T	F	T
3	p	F	F	F	T	F
4	d	F	F	F	F	T

To build the second table for finding the minimum cuts, we can iterate through the first table built for `isPalindrome()`. At any step, if we get a palindrome, we can cut the string there. Which means minimum cuts will be one plus the cuts needed for the remaining string.

Here is the code for the bottom-up approach:



```java

class Solution {

  public int findMPPCuts(String st) {
    // isPalindrome[i][j] will be 'true' if the string from index 'i' to index 'j' is a
    boolean[][] isPalindrome = new boolean[st.length()][st.length()];

    // every string with one character is a palindrome
    for (int i = 0; i < st.length(); i++)
      isPalindrome[i][i] = true;

    // populate isPalindrome table
    for (int startIndex = st.length() - 1; startIndex >= 0; startIndex--) {
      for (int endIndex = startIndex + 1; endIndex < st.length(); endIndex++) {
        if (st.charAt(startIndex) == st.charAt(endIndex)) {
          // if it's a two character string or if the remaining string is a palindrome t
          if (endIndex - startIndex == 1 || isPalindrome[startIndex + 1][endIndex - 1])
            isPalindrome[startIndex][endIndex] = true;
          }
        }
      }
```

You must run your code first

The time and space complexity of the above algorithm is $O(n^2)$, where 'n' is the length of the input string.

← Previous
Palindromic Partitioning

Next →
Longest Common Substring

☐ Mark as Completed